Project 2 Wiki

Project 02: Implementing a Simple Kernel-level Thread

1. Design (설계)

1.1 프로젝트 개요

- xv6 운영체제에서 커널 레벨 쓰레드의 간소화된 버전 구현
- 프로세스 내에서 여러 실행 컨텍스트를 동시에 유지하면서 주소 공간과 파일 디스크립터 공유

1.2 구현 목표

- [ ] `clone()` 시스템 콜 구현
- [ ] `join()` 시스템 콜 구현
- [ ] `thread_create()` 사용자 라이브러리 함수 구현
- [ ] `thread_join()` 사용자 라이브러리 함수 구현
- [ ] 기존 시스템 콜들의 쓰레드 호환성 확보

1.3 설계 접근 방법

1.3.1 데이터 구조 설계

`struct proc` 확장 (proc.h)

```
struct proc {
  // 기존 필드들...
  struct proc *main_thread;  // 메인 쓰레드를 가리키는 포인터
  int isThread;              // 쓰레드 여부를 나타내는 플래그 (0: 프로세스, 1: 쓰레드)
  uint64 trapframe_va;       // 쓰레드별 고유한 trapframe 가상 주소
};
```

쓰레드 그룹 개념

- **Tgid (Thread Group ID)**: 메인 쓰레드의 PID로 같은 그룹임을 식별

- **Pid**: 쓰레드 고유 ID
- **Parent**: 부모 프로세스 (모든 쓰레드가 동일한 부모를 가짐)
- **Main**: 같은 프로세스 그룹 내 메인 쓰레드 (`main_thread` 포인터로 관리)

1.3.2 메모리 관리 설계

공유 자원

- **주소 공간(pagetable, sz)**: 모든 쓰레드가 동일한 페이지 테이블 공유

```
np→pagetable = p→pagetable;  // 페이지 테이블 공유np→sz = p→sz;
// 프로세스 크기 공유
```

비공유 자원

- **레지스터 값**: 각 쓰레드별 독립적인 실행 컨텍스트
- **사용자 스택**: 쓰레드 생성 시 사용자가 제공한 스택 사용
- **파일 디스크립터(현재는 공유하지만 이번 과제에서는 비공유로 설계)**
- **trapframe_va**: 쓰레드별 고유한 가상 주소로 컨텍스트 분리

trapframe_va 설계

```
np→trapframe_va = TRAPFRAME - (np→pid * PGSIZE);
```

- 유저 영역에서 쓰레드의 상태를 쓰기 위해 필요
- 쓰레드별 컨텍스트 분리를 위한 핵심 메커니즘
- 각 쓰레드가 고유한 trapframe 가상 주소를 가져 독립적인 실행 상태 유지

1.3.3 스케줄링 설계

- xv6의 기본 라운드 로빈 스케줄러 활용
- 쓰레드와 프로세스를 동일하게 스케줄링 (차별화 없음)
- `struct proc` 배열에서 `state == RUNNABLE` 인 모든 항목을 동등하게 처리

1.3.4 프로세스 생성 및 종료 설계

fork() 동작

- 새로운 프로세스는 자기 자신이 메인 쓰레드가 됨

- `np->main_thread = np;`

clone() 동작

- 새로운 쓰레드는 부모의 메인 쓰레드를 참조
- `np->main_thread = p;` (호출한 프로세스/쓰레드의 메인 쓰레드)

exit() 동작 차별화

- 메인 쓰레드 종료: 전체 프로세스 그룹 종료
- 일반 쓰레드 종료: 해당 쓰레드만 종료

1.3.5 시스템 콜 호환성 설계

growproc() 메모리 확장

```
for(struct proc *pp = proc; pp < &proc[NPROC]; pp++)
  if(pp->state != UNUSED && pp->pagetable == p->pagetable && pp->main_
thread == p)
    pp->sz = sz;
```

- 같은 페이지 테이블을 공유하는 모든 쓰레드의 size 동기화

kill() 전체 프로세스 종료

```
int target_main_thread = target->main_thread->pid;
for(struct proc *pp = proc; pp < &proc[NPROC]; pp++){
  if(pp->state != UNUSED && pp->main_thread->pid == target_main_thread)
{
    pp->killed = 1;
  }
}
```

- 하나의 쓰레드를 kill하면 같은 그룹의 모든 쓰레드 종료

---

2. Implementation (구현)

2.1 핵심 함수 구현 (join, clone, freeproc, fork, kill, exec)

2.1.1 System Call Wrappers (syscall.c)

sys_clone() 시스템 콜 래퍼

```
uint64 sys_clone(void) {
  void (*fcn)(void*, void*);
  void *arg1, *arg2, *stack;

  argaddr(0, (uint64*)&fcn);   // 쓰레드 시작 함수
  argaddr(1, (uint64*)&arg1);  // 첫 번째 인자
  argaddr(2, (uint64*)&arg2);  // 두 번째 인자
  argaddr(3, (uint64*)&stack); // 사용자 스택 주소

  return clone(fcn, arg1, arg2, stack);
}
```

sys_join() 시스템 콜 래퍼

```
uint64 sys_join(void) {
  void *stack;
  argaddr(0, (uint64*)&stack);  // 스택 주소를 받을 포인터
  return join(stack);
}
```

2.1.2 clone() 함수 구현 상세

핵심 구현 전략:

1. **주소 공간 공유**: 새 쓰레드가 부모와 동일한 페이지 테이블 사용
2. **독립적인 실행 컨텍스트**: 각 쓰레드별 고유한 trapframe_va 할당
3. **파일 디스크립터 복사**: 쓰레드 생성 시점의 부모 상태 복사

```
int clone(void(*fcn)(void*, void*), void *arg1, void *arg2, void *stack)
{
  int i, pid;
  struct proc *np;
  struct proc *p = myproc();

  if((np = allocproc()) == 0){
    return -1;
  }
```

```
  // 핵심: 주소 공간 공유 (메모리 복사하지 않음)
  np->pagetable = p->pagetable;
  np->sz = p->sz;

  // 레지스터 상태 복사 후 쓰레드별 설정
  *(np->trapframe) = *(p->trapframe);
  np->isThread = 1;                    // 쓰레드 플래그 설정
  np->trapframe->epc = (uint64)fcn;       // 시작 함수 주소
  np->trapframe->a0 = (uint64)arg1;       // 첫 번째 인자
  np->trapframe->a1 = (uint64)arg2;       // 두 번째 인자
  np->trapframe_va = TRAPFRAME - (np->pid * PGSIZE); // 고유 trapframe 주
소
  np->trapframe->sp = (uint64)stack + PGSIZE;       // 스택 포인터 설정
  np->trapframe->ra = 0;                  // 리턴 주소 초기화

  // 쓰레드별 trapframe 가상 주소 매핑
  if(mappages(np->pagetable, np->trapframe_va, PGSIZE,
              (uint64)np->trapframe, PTE_R | PTE_W) < 0){
    kfree((void*)np->trapframe);
    freeproc(np);
    release(&np->lock);
    return -1;
  }

  // 커널 스택 컨텍스트 설정
  np->context.sp = np->kstack + PGSIZE;
  np->context.ra = (uint64)forkret;

  // 파일 디스크립터 복사
  for(i = 0; i < NOFILE; i++)
    if(p->ofile[i])
      np->ofile[i] = filedup(p->ofile[i]);
  np->cwd = idup(p->cwd);

  safestrcpy(np->name, p->name, sizeof(p->name));

  // 쓰레드 그룹 설정 (핵심!)
  np->main_thread = p;  // 현재 프로세스를 메인 쓰레드로 참조
```

```
  pid = np->pid;
  release(&np->lock);

  acquire(&wait_lock);
  np->parent = p;
  release(&wait_lock);

  acquire(&np->lock);
  np->state = RUNNABLE;
  release(&np->lock);

  return pid;
}
```

2.1.3 join() 함수 구현 상세

핵심 구현 전략:

1. **자식 쓰레드 식별**: `pp->main_thread == p && pp->isThread` 조건
2. **ZOMBIE 상태 대기**: 쓰레드가 완전히 종료될 때까지 대기
3. **스택 주소 반환**: 사용자가 할당한 스택 주소를 안전하게 반환

```
int join(void** stack)
{
  struct proc *pp;
  int havekids, pid;
  struct proc *p = myproc();

  acquire(&wait_lock);

  for(;;){
    havekids = 0;

    // 프로세스 테이블에서 내 쓰레드들 검색
    for(pp = proc; pp < &proc[NPROC]; pp++){
      // 조건: 내가 메인 쓰레드이고, pp가 내 쓰레드인 경우
      if(pp->main_thread == p && pp->isThread){
```

```
      acquire(&pp->lock);
      havekids = 1;

      if(pp->state == ZOMBIE){
        // 종료된 쓰레드 발견

        // 스택 주소 계산 및 사용자 공간으로 복사
        uint64 stack_addr = pp->trapframe->sp - PGSIZE;
        if(stack != 0 && copyout(pp->pagetable, (uint64)stack,
                              (char*)&stack_addr, sizeof(uint64)) < 0) {
          release(&pp->lock);
          release(&wait_lock);
          return -1;
        }

        pid = pp->pid;
        freeproc(pp);  // 쓰레드 자원 정리
        release(&pp->lock);
        release(&wait_lock);
        return pid;
      }
      release(&pp->lock);
    }
  }

  // 대기할 자식 쓰레드가 없거나 현재 프로세스가 kill된 경우
  if(!havekids || killed(p)){
    release(&wait_lock);
    return -1;
  }

  // 자식 쓰레드 종료까지 대기
  sleep(p, &wait_lock);
 }
}
```

2.1.4 freeproc() 함수 수정

핵심 개선사항:

1. **쓰레드별 trapframe_va 해제**: 독립적인 가상 주소 매핑 정리
2. **페이지 테이블 공유 처리**: 마지막 프로세스인 경우에만 해제
3. **안전한 자원 정리**: 메모리 누수 방지

```
static void freeproc(struct proc *p)
{
  // 쓰레드인 경우 독립적인 trapframe_va 매핑 해제
  if(p->trapframe_va && p->isThread){
    pte_t *pte = walk(p->pagetable, p->trapframe_va, 0);
    if(pte && (*pte & PTE_V)) {
      uvmunmap(p->pagetable, p->trapframe_va, 1, 0);
    }
  }

  // 같은 페이지 테이블을 사용하는 다른 프로세스가 있는지 확인
  int target = 1;
  for(struct proc *pp = proc; pp < &proc[NPROC]; pp++){
    if(pp == p) continue;
    if(pp->state == UNUSED) continue;
    if(pp->main_thread == p->main_thread) continue;
    target = 0;  // 다른 프로세스가 있으면 페이지 테이블 해제하지 않음
    break;
  }

  // trapframe 물리 메모리 해제
  if(p->trapframe)
    kfree((void*)p->trapframe);

  // 마지막 프로세스인 경우에만 페이지 테이블 해제
  if (target && p->pagetable)
    proc_freepagetable(p->pagetable, p->sz);

  // 프로세스 구조체 초기화
  p->trapframe = 0;
  p->trapframe_va = 0;
  p->pagetable = 0;
```

```
  p->sz = 0;
  p->pid = 0;
  p->parent = 0;
  p->name[0] = 0;
  p->chan = 0;
  p->killed = 0;
  p->xstate = 0;
  p->state = UNUSED;
  p->main_thread = 0;
  p->isThread = 0;
}
```

2.1.5 fork() 함수 수정

쓰레드 지원을 위한 주요 변경사항:

```
// fork() 함수 끝부분에 추가
acquire(&wait_lock);
np->parent = p;
np->main_thread = np;  // 새 프로세스는 자기 자신이 메인 쓰레드
release(&wait_lock);
```

특징:

- 기존 fork() 로직은 그대로 유지 (메모리 복사)
- 새로운 프로세스는 독립적인 메인 쓰레드가 됨

2.1.6 kill() 함수 수정

프로세스 그룹 단위 종료 구현:

```
int kill(int pid)
{
  struct proc *p;
  struct proc *target = 0;

  // 대상 프로세스/쓰레드 찾기
  for(p = proc; p < &proc[NPROC]; p++){
    acquire(&p->lock);
```

```
    if(p->pid == pid){
      target = p;
      break;
    }
    release(&p->lock);
  }

  if(target == 0) {
    return -1;
  }

  // 메인 쓰레드 ID 저장
  int target_main_thread = target->main_thread->pid;
  release(&target->lock);

  // 같은 프로세스 그룹의 모든 쓰레드 종료
  for(struct proc *pp = proc; pp < &proc[NPROC]; pp++){
    acquire(&pp->lock);
    if(pp->state != UNUSED && pp->main_thread->pid == target_main_threa
d){
      pp->killed = 1;
      if(pp->state == SLEEPING) {
        pp->state = RUNNABLE;
      }
    }
    release(&pp->lock);
  }
  return 0;
}
```

핵심 특징:

- 하나의 쓰레드를 kill해도 전체 프로세스 그룹이 종료
- `main_thread->pid` 를 기준으로 그룹 식별

2.1.7 exec() 함수 수정

모든 쓰레드 정리 후 새 프로세스 시작:

```
int exec(char *path, char **argv)
{
  // ... (기존 변수 선언)

  begin_op();

  // 1. 같은 그룹의 모든 쓰레드에게 종료 신호
  for (struct proc *pp = proc; pp < &proc[NPROC]; pp++) {
    if (pp == p) continue;
    if (pp->state == UNUSED) continue;
    if (pp->main_thread == p->main_thread) {
      acquire(&pp->lock);
      pp->killed = 1;
      if (pp->state == SLEEPING)
        pp->state = RUNNABLE;
      release(&pp->lock);
    }
  }

  // 2. 모든 쓰레드가 종료될 때까지 대기
  int alive;
  do{
    alive = 0;
    for(struct proc *pp = proc; pp < &proc[NPROC]; pp++){
      if(pp == p) continue;
      if(pp->main_thread == p->main_thread &&
         pp->state != UNUSED && pp->state != ZOMBIE){
        alive = 1;
      }
    }
    if(alive) yield();
  }while(alive);

  // ... (기존 exec 로직)

  // 3. 새 프로세스는 자기 자신이 메인 쓰레드
  p->main_thread = p;
```

```
  return argc;

  // ... (에러 처리)
}
```

핵심 특징:

- exec 호출 시 같은 그룹의 모든 쓰레드 종료
- `yield()`를 사용하여 쓰레드들이 정리될 시간 확보
- 새 프로그램은 단일 프로세스로 시작

2.2 사용자 라이브러리 함수 구현

2.2.1 thread_create() 및 thread_join()

파일 위치: `user/thread.h`, `user/thread.c`

```
// thread_create: 쓰레드 생성 및 스택 할당
int thread_create(void (*start_routine)(void *, void *), void *arg1, void *arg
2);

// thread_join: 쓰레드 종료 대기 및 스택 해제
int thread_join();
```

구현 특징:

- `thread_create()`: 스택 메모리 할당 후 `clone()` 시스템 콜 호출
- `thread_join()`: `join()` 시스템 콜 호출 후 스택 메모리 해제
- ULIB에 `thread.o` 추가하여 라이브러리 지원

2.3 기존 시스템 콜 수정

2.3 기타 시스템 콜 수정

2.3.1 growproc() 함수 - 메모리 동기화

```
int growproc(int n)
{
  uint64 sz;
```

```
  struct proc *p = myproc();
  acquire(&wait_lock);  // 동기화를 위한 락
  sz = p->sz;

  if(n > 0){
    if((sz = uvmalloc(p->pagetable, sz, sz + n, PTE_W)) == 0) {
      return -1;
    }
  } else if(n < 0){
    sz = uvmdealloc(p->pagetable, sz, sz + n);
  }

  // 같은 페이지 테이블을 공유하는 모든 쓰레드의 size 업데이트
  for(struct proc *pp = proc; pp < &proc[NPROC]; pp++)
    if(pp->state != UNUSED && pp->pagetable == p->pagetable && pp->main
_thread == p)
      pp->sz = sz;

  release(&wait_lock);
  return 0;
}
```

2.3.2 exit() 함수 - 쓰레드별 종료 처리

```
// exit() 함수의 wakeup 부분 수정
if(p->isThread && p->main_thread) {
  wakeup(p->main_thread);  // 메인 쓰레드 깨우기
} else {
  wakeup(p->parent);       // 부모 프로세스 깨우기
}
```

2.3.3 allocproc() 함수 - 초기화 추가

```
// allocproc() 함수 끝부분에 추가
p->main_thread = 0;
p->isThread = 0;
```

기타 수정사항:

- **sleep(), pipe()**: 기존 동작 유지 (별도 수정 불필요)
- **scheduler()**: 쓰레드와 프로세스를 동일하게 스케줄링

2.4 주요 설계 특징

2.4.1 trapframe_va를 통한 컨텍스트 분리 (핵심 설계)

```
np→trapframe_va = TRAPFRAME - (np→pid * PGSIZE);
```

설계 목적:

- 유저 영역에서 쓰레드의 상태를 독립적으로 관리
- 각 쓰레드가 고유한 trapframe 가상 주소를 가져 실행 컨텍스트 분리
- 동일한 페이지 테이블을 공유하면서도 독립적인 실행 상태 유지

2.4.2 프로세스 그룹 관리

그룹 식별 메커니즘:

- `main_thread` 포인터로 같은 프로세스 그룹 식별
- 메인 쓰레드: `main_thread == 자기 자신` (fork에서 설정)
- 일반 쓰레드: `main_thread == 생성한 프로세스` (clone에서 설정)

그룹 단위 작업 예시:

```
// kill, exec, growproc 등에서 사용되는 패턴
for(struct proc *pp = proc; pp < &proc[NPROC]; pp++){
  if(pp→state != UNUSED && pp→main_thread == target_main_thread){
    // 그룹 단위 작업 수행
  }
}
```

2.4.3 메모리 관리 전략

공유 자원:

- 페이지 테이블 (`pagetable`), 프로세스 크기 (`sz`)
- 주소 공간 전체 (코드, 데이터, 힙 영역)

독립 자원:

- 사용자 스택 (쓰레드 생성 시 사용자 제공)
- trapframe 및 trapframe_va (실행 컨텍스트)
- 파일 디스크립터 테이블 (생성 시점에 복사)

2.4.4 안전한 자원 해제

freeproc()의 스마트 해제:

- 쓰레드별 trapframe_va 매핑만 개별 해제
- 페이지 테이블은 마지막 프로세스/쓰레드에서만 해제
- 메모리 누수 방지와 공유 자원 보호 균형

2.5 동기화 및 경합 조건 처리

2.5.1 사용된 동기화 메커니즘

wait_lock 사용:

```
// growproc()에서 메모리 확장 시 동기화
acquire(&wait_lock);
// 메모리 할당/해제 작업
for(struct proc *pp = proc; pp < &proc[NPROC]; pp++)
  if(pp->state != UNUSED && pp->pagetable == p->pagetable && pp->main_
thread == p)
    pp->sz = sz;
release(&wait_lock);
```

프로세스별 락 (p->lock) 사용:

```
// 프로세스 상태 변경 시 항상 락 획득
acquire(&p->lock);
p->state = RUNNABLE;
release(&p->lock);
```

2.5.2 경합 조건 방지 전략

1. 프로세스 테이블 접근 동기화

- 각 `struct proc`에 대한 개별 락 사용
- 프로세스 상태 변경 시 해당 프로세스 락 필수 획득

2. 메모리 할당/해제 동기화

- `wait_lock`을 사용하여 여러 쓰레드의 동시 메모리 변경 방지
- 같은 페이지 테이블을 공유하는 모든 쓰레드의 size 동기화

3. 파일 시스템 접근 동기화

- `filedup()`, `fileclose()` 등은 내부적으로 동기화 처리
- 파일 객체의 참조 카운팅으로 안전한 공유

2.5.3 데드락 방지

락 순서 규칙:

1. `wait_lock` → `p->lock` 순서로 획득
2. 여러 프로세스 락이 필요한 경우 PID 순서로 획득
3. 락 보유 시간 최소화

예시:

```
// join() 함수에서의 올바른 락 순서
acquire(&wait_lock);        // 1. wait_lock 먼저
for(pp = proc; pp < &proc[NPROC]; pp++){
  acquire(&pp->lock);       // 2. 개별 프로세스 락
  // 작업 수행
  release(&pp->lock);
}
release(&wait_lock);
```

2.5.4 원자성 보장

- `allocpid()`: `pid_lock`으로 PID 할당 원자성 보장
- `freeproc()`: 프로세스 정리 과정의 원자성 보장
- 상태 변경과 스케줄링의 원자성 보장

---

3. Results (결과)

3.1 컴파일 과정

```
# 컴파일 명령어 및 결과
$ make qemu
```

3.2 테스트 프로그램 실행 결과

```
Thread 1 size = 0x0000000000035000
Thread 2 size = 0x0000000000035000
Thread 3 size = 0x0000000000035000
Thread 4 size = 0x0000000000035000
Thread 0 sbrk: free memory
usertrap(): unexpected scause 0xf pid=24
            sepc=0xedc stval=0x8
usertrap(): unexpected scause 0xf pid=25
            sepc=0xedc stval=0x8
usertrap(): unexpected scause 0xf pid=26
            sepc=0xedc stval=0x8
usertrap(): unexpected scause 0xf pid=27
            sepc=0xedc stval=0x8
usertrap(): unexpected scause 0xf pid=28
            sepc=0xedc stval=0x8
TEST#4 Passed

[TEST#5]
usertrap(): unexpected scause 0xf pid=29
            sepc=0xedc stval=0x8
TEST#5 Passed

[TEST#6]
usertrap(): unexpected scause 0xf pid=30
            sepc=0xedc stval=0x8
TEST#6 Passed

All tests passed. Great job!!
$ QEMU: Terminated
```

---

4. Troubleshooting (문제 해결)

4.1 발생한 문제점들

4.1.1 문제 1: test2 실행 시 usertrap 발생으로 즉시 종료

증상:

- test2 테스트 실행 후 usertrap이 발생하여 프로그램이 바로 종료됨
- 쓰레드가 정상적으로 생성되지 않고 트랩으로 인한 비정상 종료

원인 분석:

- 쓰레드 스택 크기 부족으로 인한 스택 오버플로우
- `thread_create()` 에서 할당한 스택 크기가 쓰레드 실행에 불충분
- 초기 1 페이지(4KB) 스택으로는 쓰레드 함수 실행 시 메모리 부족 발생

해결 방법:

- 쓰레드 스택 크기를 2배(2 페이지, 8KB)로 증가

```
// 기존 코드 (1 페이지)
void *stack = malloc(PGSIZE);

// 수정된 코드 (2 페이지)
void *stack = malloc(2 * PGSIZE);
```

결과:

- 스택 오버플로우 문제 해결
- test2 및 기타 쓰레드 테스트 정상 동작

주요 코드 변경사항:

1. **proc.h 구조체 확장**

```
struct proc {
  // 기존 필드들...
  uint64 trapframe_va;  // 쓰레드별 고유 trapframe 가상 주소 추가
};
```

1. **freeproc() 함수 수정**

```
static void freeproc(struct proc *p)
{
  // trapframe_va 초기화 추가
  p->trapframe_va = 0;
  // 기타 필드 초기화...
}
```

1. **proc_pagetable() 함수 수정**

```
pagetable_t proc_pagetable(struct proc *p)
{
  // trapframe_va 설정 로직 추가
  if(mappages(pagetable, p->trapframe_va = TRAPFRAME, PGSIZE,
              (uint64)(p->trapframe), PTE_R | PTE_W) < 0){
    // 에러 처리
  }
}
```

1. **trampoline.S 어셈블리 코드 수정**

```
# 사용자 모드 복귀 시 sscratch 레지스터 활용
# trapframe_va를 직접 참조하지 않고 레지스터를 통해 간접 접근
```

4.1.3 문제 3: 메모리 매핑 충돌

증상:

- 여러 쓰레드 생성 시 trapframe_va 주소 충돌
- 쓰레드별 독립적인 컨텍스트 유지 실패

원인 분석:

- 동일한 trapframe_va 주소를 여러 쓰레드가 사용하려 시도
- PID 기반 주소 계산에서 오류 발생

해결 방법:

```
// clone() 함수에서 고유한 trapframe_va 할당
np->trapframe_va = TRAPFRAME - (np->pid * PGSIZE);
```

```
// 각 쓰레드별로 고유한 가상 주소 보장
if(mappages(np→pagetable, np→trapframe_va, PGSIZE,
            (uint64)np→trapframe, PTE_R | PTE_W) < 0){
  // 매핑 실패 시 정리
}
```

---

5. Conclusion (결론)

5.1 구현 완성도

- 완료된 기능: 스레드 생성 및 테스트 시 Great job 문구 출력 확인
- 부분 완료된 기능: freeproc 시 로직 다소 미흡

5.2 학습 성과

- 운영체제 커널 프로그래밍 경험 획득
- 쓰레드와 프로세스의 차이점 실제 구현을 통한 이해
- 시스템 콜 인터페이스 설계 및 구현 경험

5.3 향후 개선 방향

- Usertrap 안 뜨고 작동하도록 수